

Prueba técnica — Fullstack Engineer

Cómo leer esta prueba (leé esto primero)

Asumimos que vas a usar IA. Está perfecto y lo esperamos. **Que funcione es el piso, no el techo.** No es un test de "¿anda?". Evaluamos:

1. Cómo modelaste el dominio y separaste responsabilidades (back y front).
2. Tu criterio: qué decidiste, qué descartaste, qué dejaste afuera a propósito.
3. Que puedas **explicar el porqué** y mostrar **dónde corregiste a la IA**.

Una solución que anda pero que no podés defender puntúa por debajo de una más simple defendida con criterio. El scope es chico **a propósito**: la dificultad está en hacerlo bien y sostenerlo, no en hacer mucho.

Tiempo: 6–8 horas efectivas. Entrega: hasta 7 días. Si te lleva más, **pará** y contá en el README qué harías con más tiempo. Cronometrar tu propio scope es parte de lo que miramos.

Contexto

Software de accountability y compliance para founders: que las empresas no pierdan de vista sus obligaciones (vencimientos, presentaciones, documentación). Dominio de alto cuidado: una fecha mal calculada, un dato sensible filtrado o un cambio sin registrar son errores caros. Dominio ficticio y de conocimiento público — no esperamos que sepas de compliance, todo lo que necesitás está acá.

El proyecto: "Compliance Obligations Tracker"

Seguir las obligaciones de compliance de una empresa: qué presentar, cuándo vence, en qué estado, con qué documentación.

Modelo — Obligation

`id` · `type` (annual_report | franchise_tax | boi_report | registered_agent_renewal) · `title`,
`description` · `status` (pending | in_progress | submitted | done) · `dueDate` · `owner`
(obligatorio) · `requiresDocument` (bool) + doc adjunto (puede ser mock) · `companyTaxId`
(DATO SENSIBLE).

Reglas de dominio (la chicha — esto es lo que evaluamos)

Máquina de estados — fuente de verdad del BACKEND:

- pending → in_progress
- in_progress → submitted | pending
- submitted → done | in_progress (rechazo/rework)
- done → in_progress (reabrir)
- Transición inválida → la API la rechaza; **no se persiste por ninguna vía.**

Invariante documento-gated: si `requiresDocument`, NO puede pasar a `submitted` sin documento adjunto. Regla del backend, no del form.

overdue es **DERIVADO**, no un estado: vencida = `dueDate` pasó y no está submitted/done. Pensá dónde vive ese cálculo (lo preguntamos en la defensa).

Dato sensible: `companyTaxId` se guarda completo, se muestra enmascarado (`••••6789`) y **NUNCA** se loguea.

Trazabilidad (audit trail): cada cambio de estado registrado (de→a, cuándo). El detalle muestra el historial.

Concurrencia: dos requests cambian el estado a la vez — tu diseño tiene que contemplarlo y lo explicás (versión / optimistic lock / transacción).

Requisitos funcionales

Backend:

- CRUD + endpoint de cambiar-estado; validación server-side.
- Máquina de estados + invariante doc-gated + cálculo de `overdue` + audit, **todo en una capa de dominio aislada de HTTP y de la base de datos.** La regla no va en el handler.
- `companyTaxId` enmascarado en lecturas y fuera de logs.
- Persistencia real (Postgres recomendado con docker-compose; SQLite OK).
- Modelo de error consistente (HTTP, 404).

Frontend:

- **Dashboard:** KPIs (total / por estado / vencidas / próximas a vencer), filtro, lista ordenada por `dueDate` con resalte de vencidas.
- **Detalle:** todos los campos + `taxId` enmascarado + historial + transiciones válidas disponibles, con el botón de `submitted` **bloqueado** si falta documento.
- **Crear / editar** con validación.
- **i18n es/en.**
- Consume la API. **No dupliques el dominio en el front** — transiciones válidas y bloqueo vienen del backend.

- Loading y error de la API reflejados en la UI.

Stack (obligatorio)

- **Backend:** FastAPI + Pydantic + PostgreSQL (SQLite OK). Capas: rutas / servicios / acceso a datos / dominio. *(Alternativa Node/TS OK — la consigna funcional no cambia.)*
- **Frontend:** Next.js (App Router) + React + TS strict (sin **any**) + Tailwind (sin librería pesada). Server Components por defecto; Server Actions para mutaciones.

Arquitectura esperada (lo que más miramos)

Separación deliberada. **Backend:** dominio (estados, invariantes, **overdue**, audit) aislado de HTTP y de datos — la regla no va en el handler. **Frontend:** por capas/features, UI primitiva reutilizable, límites server/client claros.

No funcionales

TS strict + Pydantic honesto · validación server-side · invariantes inviolables · dato sensible enmascarado y fuera de logs · **testing de comportamiento en ambas capas** (back: estados + invariante doc-gated + un endpoint; front: un flujo) · errores y loading de API a UI · a11y básica · commits limpios.

DECISIONS.md (peso alto — esto se defiende)

- Arquitectura (back + front): cómo aislaste el dominio, alternativas descartadas, qué dejaste afuera y por qué.
- Contrato de la API (método/ruta/req/resp/errores). OpenAPI cuenta doble.
- Dónde vive **overdue**, cómo manejas concurrencia, cómo tratás el dato sensible — **con el porqué de cada uno**.
- Uso de IA explícito: dónde te ayudó y **dónde la corregiste o rechazaste** (ejemplos concretos).

Nota: la migración de un legacy NO va en el DECISIONS.md. La trabajamos en vivo en la defensa.

La defensa técnica (después de la entrega)

Vas a recorrer y justificar tu arquitectura, defender trade-offs, contar dónde la IA te llevó mal y responder "¿y si...?". **Prepará la entrega para defenderla** — es la parte que más pesa.

Stretch (opcionales)

Recordatorios/próximas a vencer · subida real de documentos · paginación/búsqueda · CI (lint+test ambas capas) · auth simple · logs estructurados.

Qué NO esperamos

Pixel-perfect · cobertura 100% · todos los stretch · infra compleja. **Poco y sólido.**

Entregables

Repo (back + front) · README (qué hiciste, cómo levantar todo, qué dejaste afuera, qué harías con más tiempo) · DECISIONS.md. **PLUS:** Puedes entregar el proyecto en producción

Rúbrica

Dimensión	Peso
Arquitectura y diseño (back + front, modelado de dominio)	25%
Decisiones y capacidad de explicar el porqué (DECISIONS.md + defensa)	20%
Calidad de código y tipos (TS + Pydantic)	15%
Testing en ambas capas (comportamiento)	15%
Integridad, validación, dato sensible y concurrencia	15%
Manejo de fechas/ overdue y correctitud funcional e integración	10%

Correctitud funcional = piso, no diferencial.